Debugging

2026.03.26

SWPP Practice Session

Seunghyeon Nam

Typical Bugfix Process

- Notice an error
- Narrow down the line that causes the error
  - If the program crashes, look for the assertion or invalid pointer
  - If the program yields wrong output, look for the output variable
- Traceback to the line that started to go wrong

Narrowing Down the Line

- How do you locate the exact line that crashes?
  - Guess the location
  - Insert some `std::cout` or `std::cerr` all over the code
  - Rebuild
  - Look for the last printed message
  - Repeat until you actually pinpoint the location

Narrowing Down the Line

- This is horribly inefficient!
  - Taking a wild guess in a large codebase purely depends on luck
  - Rebuilding a large codebase may take minutes or even hours
  - And you have to repeat it until you actually find the bug
- Locating a single bug may already take hours or days

Traceback

- Most of the errors cannot be fixed locally
  - It is likely that the code that ‘triggers’ the error is not a bug
  - The code that ‘leads to’ the error is the real verdict
  - But these two are usually far away from each other…
- You have to locate the code that first went wrong
  - Narrow down, take a step back, narrow down, again and again

Debugger to the Rescue

- Debugger can control the execution of your program
  - Line by line
  - In and out of function
  - Pause on assertion, throw, catch, breakpoint

Debugger to the Rescue

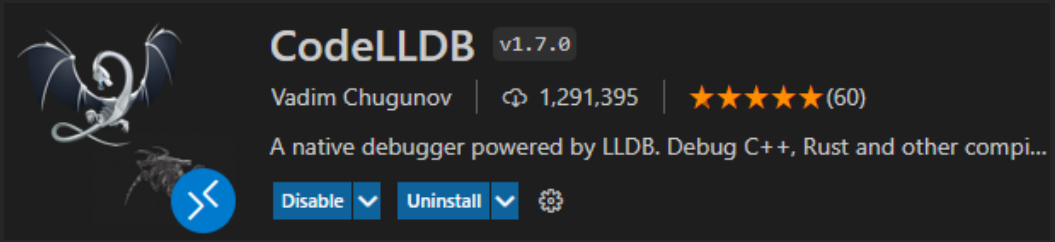
- Debugger can expose the execution context of your program
  - Call stack
  - Local/global variables and values

Using the Debugger

- LLDB: LLVM debugger
  - You have to enable LLDB project when building the LLVM
  - Already included if you built the LLVM using class repo script
- You must build your program with clang
- You must build your program in debug mode

Using the Debugger

- We'll use vscode extension for convenience
  - CodeLLDB



- LLDB directory should be added to your PATH
  - What is PATH?

File Edit Selection View Go Run Terminal Help

inst.cpp - swpp-compiler [S /isual Studio Code

RUN AND DEBUG

RUN

Run and Debug

To customize Run and Debug create a launch.json file.

Show all automatic debug configurations.

To learn more about launch.json, see Configuring C/C++ debugging.

inst.cpp | lib.cpp 8

src > lib.cpp > {} `anonymous-namespace' > InputF

Select environment

C++ (GDB/LLDB)

C++ (Windows)

LLDB

Install an extension for C++...

Select LLDB from the options

```
19    std::string message;
20
21  public:
22    InputFileError(sc::parser::ParserError &
23      using namespace std::string_literals;
24      message = "invalid input file\n"s.append(__err.what());
25    }
26
27    InputFileError(fs::FilesystemError &&__err) noexcept {
28      using namespace std::string_literals;
29      message = "invalid input file\n"s.append(__err.what());
30    }
31
32    const char *what() const noexcept { return message.c_str(); }
33  };
34
    You, last month | 1 author (You)
35  class OutputFileError : public Error<OutputFileError> {
36  private:
37    std::string message;
38
39  public:
40    OutputFileError(const std::string_view __message) {
41      using namespace std::string_literals;
42      message = "invalid output file\n"s;
43      message.append(__message);
44    }
45    const char *what() const noexcept { return message.c_str(); }
46  };
47
48  Result<std::string, InputFileError>
49  readFile(const std::string_view __filename) {
50    auto read_result = fs::readFile(__filename);
51    return decltype(read_result)::mapErr<InputFileError>(
52        std::move(read_result),
53        [](auto &&err) { return InputFileError(std::move(err)); });
```

ssembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
    locInst
297                                  MallocInst(const GeneralRegister __target,
298                                             ValueTy &&__size) noexcept
      : AbstractInst(), target(__target), size(std::move(__size)) {}
300 MallocInst MallocInst::create(const GeneralRegister __target,
301                               ValueTy &&__size) noexcept {
302   return MallocInst(__target, std::move(__size));
303 }
304
305 std::string MallocInst::getAssembly() const noexcept {
306   return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307 }
308         You, 2 months ago • Init …
309 //-----------------------------------------------------
310 // class FreeInst
311 //-----------------------------------------------------
312 FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313     : AbstractInst(), ptr(std::move(__ptr)) {}
314
315 FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316   return FreeInst(std::move(__ptr));
317 }
318
319 std::string FreeInst::getAssembly() const noexcept {
320   return joinTokens(collectOpTokens("free"s, ptr));
321 }
322
323 //-----------------------------------------------------
324 // class LoadInst
325 //-----------------------------------------------------
326 LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                    ValueTy &&__ptr) noexcept
328     : AbstractInst(), target(__target), size(__size), ptr(std::move(__ptr)) {}
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

zsh

swpp-compiler main

MODULES

SSH: main* 8 0 0 CMake: [Debug]: Ready [Clang 14.0.0 x86_64-unknown-linux-gnu] Build [all] [swpp-compiler] You, 2 months ago Ln 308, Col 1 Spaces: 2 UTF-8 LF C++ Linux

File Edit Selection View Go Run Terminal Help — lib.cpp - swpp-compiler — Visual Studio Code

RUN AND DEBUG: RUN

Run and Debug

To customize Run and Debug create a launch.json file.

Show all automatic debug configurations.

To learn more about launch.json, see Configuring C/C++ debugging.

emitter.cpp | lib.cpp 8

src > lib.cpp > {} `anonymous-namespace' > InputFileError > InputFileError(sc::parser::ParserError &&)

```
19    std::string message;
20
21  public:
22    InputFileError(sc::parser::ParserError &&__err) noexcept {
23      using namespace std::string_literals;
24      message = "invalid input file\n"s.append(__err.what());
25    }        You, 2 months ago • Init …
26
27    InputFileError(fs::FilesystemError &&__err) noexcept {
28      using namespace std::string_literals;
29      message = "invalid input file\n"s.append(__err.what());
30    }
31
32    const char *what() const noexcept { return message.c_str(); }
33  };
34
    You, last month | 1 author (You)
35  class OutputFileError : public Error<OutputFileError> {
36  private:
37    std::string message;
38
39  public:
40    OutputFileError(const std::string_view __messag
41      using namespace std::string_literals;
42      message = "invalid output file\n"s;
43      message.append(__message);
44    }
45    const char *what() const noexcept { return message.c_str(); }
46  };
47
48  Result<std::string, InputFileError>
49  readFile(const std::string_view __filename) {
50    auto read_result = fs::readFile(__filename);
51    return decltype(read_result)::mapErr<InputFileError>(
52        std::move(read_result),
53        [](auto &&err) { return InputFileError(std::move(err)); });
```

inst.cpp

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
293  //-----------------------------------------------------------
294  // class MallocInst
295  //-----------------------------------------------------------
296  MallocInst::MallocInst(const GeneralRegister __target,
297                         ValueTy &&__size) noexcept
298      : AbstractInst(), target(__target), size(std::move(__size)) {}
299
300  MallocInst MallocInst::create(const GeneralRegister __target,
301                                ValueTy &&__size) noexcept {
302    return MallocInst(__target, std::move(__size));
303  }
304
305  std::string MallocInst::getAssembly() const noexcept {
306    return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307  }
308
309  //-----------------------------------------------------------
310  // class FreeInst
311  //-----------------------------------------------------------
312  eInst::FreeInst(ValueTy &&__ptr) noexcept
313      : AbstractInst(), ptr(std::move(__ptr)) {}
314
315  eInst FreeInst::create(ValueTy &&__ptr) noexcept {
316    eturn FreeInst(std::move(__ptr));
317  }
318
319  std::string FreeInst::getAssembly() const noexcept {
320    return joinTokens(collectOpTokens("free"s, ptr));
321  }
322
323  //-----------------------------------------------------------
324  // class LoadInst
325  //-----------------------------------------------------------
326  LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                     ValueTy &&__ptr) noexcept
```

You'll get this error on the first run

Visual Studio Code

Cannot start debugging because no launch configuration has been provided.

OK

PROBLEMS 8 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

swpp-compiler main

SSH: | main* | 8 0 | 0 | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build | [all] | [swpp-compiler] | You, 2 months ago | Ln 25, Col 4 | Spaces: 2 | UTF-8 | LF | C++ | Linux

File Edit Selection View Go Run Terminal Help launch.json - swpp-compiler - Visual Studio Code

RUN ... Debug

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

C++: on throw

C++: on catch

emitter.cpp | lib.cpp 8 | launch.json

.vscode > launch.json > ...

```
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "type": "lldb",
9              "request": "launch",
10             "name": "Debug",
11             "program": "${workspaceFolder}/<your program>",
12             "args": [],
13             "cwd": "${workspaceFolder}"
14         }
15     ]
16 }
```

vscode will generate this template

Add Configuration...

inst.cpp

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
293  //-----------------------------------------------------------
294  // class MallocInst
295  //-----------------------------------------------------------
296  MallocInst::MallocInst(const GeneralRegister __target,
297                         ValueTy &&__size) noexcept
298      : AbstractInst(), target(__target), size(std::move(__size)) {}
299
300  MallocInst MallocInst::create(const GeneralRegister __target,
301                                ValueTy &&__size) noexcept {
302    return MallocInst(__target, std::move(__size));
303  }
304
305  std::string MallocInst::getAssembly() const noexcept {
306    return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307  }
308
309  //-----------------------------------------------------------
310  // class FreeInst
311  //-----------------------------------------------------------
312  FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313      : AbstractInst(), ptr(std::move(__ptr)) {}
314
315  FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316    return FreeInst(std::move(__ptr));
317  }
318
319  std::string FreeInst::getAssembly() const noexcept {
320    return joinTokens(collectOpTokens("free"s, ptr));
321  }
322
323  //-----------------------------------------------------------
324  // class LoadInst
325  //-----------------------------------------------------------
326  LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                     ValueTy &&__ptr) noexcept
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

zsh

swpp-compiler main

SSH: main* 8 0 0 Debug (swpp-compiler) CMake: [Debug]: Ready [Clang 14.0.0 x86_64-unknown-linux-gnu] Build [all] [swpp-compiler] Ln 1, Col 1 Spaces: 4 UTF-8 LF JSON with Comments Linux

File Edit Selection View Go Run Terminal Help launch.json - swpp-compiler - Visual Studio Code

RUN Debug

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

C++: on throw

C++: on catch

emitter.cpp | lib.cpp 8 | launch.json

.vscode > launch.json > Launch Targets > {} Debug

```
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "type": "lldb",
9              "request": "launch",
10             "name": "Debug",
11             "program": "${workspaceFolder}/build/swpp-compiler",
12             "args": ["benchmarks/anagram/src/anagram.ll", "test.s"],
13             "cwd": "${workspaceFolder}"
14         }
15     ]
16 }
```

Fill in the program and args.
Then run the debugger

Add Configuration...

inst.cpp

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
293 //-----------------------------------------------------------
294 // class MallocInst
295 //-----------------------------------------------------------
296 MallocInst::MallocInst(const GeneralRegister __target,
297                        ValueTy &&__size) noexcept
298     : AbstractInst(), target(__target), size(std::move(__size)) {}
299
300 MallocInst MallocInst::create(const GeneralRegister __target,
301                               ValueTy &&__size) noexcept {
302   return MallocInst(__target, std::move(__size));
303 }
304
305 std::string MallocInst::getAssembly() const noexcept {
306   return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307 }
308
309 //-----------------------------------------------------------
310 // class FreeInst
311 //-----------------------------------------------------------
312 FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313     : AbstractInst(), ptr(std::move(__ptr)) {}
314
315 FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316   return FreeInst(std::move(__ptr));
317 }
318
319 std::string FreeInst::getAssembly() const noexcept {
320   return joinTokens(collectOpTokens("free"s, ptr));
321 }
322
323 //-----------------------------------------------------------
324 // class LoadInst
325 //-----------------------------------------------------------
326 LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                    ValueTy &&__ptr) noexcept
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

zsh

swpp-compiler main

SSH: main* 8 0 0 Debug (swpp-compiler) CMake: [Debug]: Ready [Clang 14.0.0 x86_64-unknown-linux-gnu] Build [all] [swpp-compiler] Ln 12, Col 69 Spaces: 4 UTF-8 LF JSON with Comments Linux

File Edit Selection View Go Run Terminal Help — eh_throw.cc - swpp-compiler - Visual Studio Code

RUN ... Debug

emitter.cpp | launch.json | eh_throw.cc

libstdc++-v3 > libsupc++ > eh_throw.cc > __cxa_throw(void *, std::type_info *, void(_GLIBCXX_CDTOR_

```
60  {
61    __cxa_refcounted_exception *header
62      = __get_refcounted_exception_header_from_obj (obj);
63    header->referenceCount = 0;
64    header->exc.exceptionType = tinfo;
65    header->exc.exceptionDestructor = dest;
66    header->exc.unexpectedHandler = std::get_unexpected ();
67    header->exc.terminateHandler = std::get_terminate ();
68    __GXX_INIT_PRIMARY_EXCEPTION_CLASS(header->exc.unwindHeader.exception_class);
69    header->exc.unwindHeader.exception_cleanup = __gxx_exception_cleanup;
70
71    return header;
72  }
73
74  extern "C" void
75  __cxxabiv1::__cxa_throw (void *obj, std::type_info *tinfo,
76          void (_GLIBCXX_CDTOR_CALLABI *dest) (void *))
77  {
78    PROBE2 (throw, obj, tinfo);
79
80    __cxa_eh_globals *const __cxa_eh_globals
81    globals->uncaught_exceptions += 1;
82    // Definitely a primary.
83    __cxa_refcounted_exception *header =
84      __cxa_init_primary_exception(obj, tinfo, dest);
85    header->referenceCount = 1;
86
87  #ifdef __USING_SJLJ_EXCEPTIONS__
88    _Unwind_SjLj_RaiseException (&header->exc.unwindHeader);
89  #else
90    _Unwind_RaiseException (&header->exc.unwindHeader);
91  #endif
92
93    // Some sort of unwinding error.  Note that terminate is a handler.
94    __cxa_begin_catch (&header->exc.unwindHeader);
```

Debugger paused at throw

inst.cpp

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
293  //-----------------------------------------------------------
294  // class MallocInst
295  //-----------------------------------------------------------
296  MallocInst::MallocInst(const GeneralRegister __target,
297                         ValueTy &&__size) noexcept
298      : AbstractInst(), target(__target), size(std::move(__size)) {}
299
300  MallocInst MallocInst::create(const GeneralRegister __target,
301                                ValueTy &&__size) noexcept {
302    return MallocInst(__target, std::move(__size));
303  }
304
305  std::string MallocInst::getAssembly() const noexcept {
306    return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307  }
308
309  //-----------------------------------------------------------
310  // class FreeInst
311  //-----------------------------------------------------------
312  FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313      : AbstractInst(), ptr(std::move(__ptr)) {}
314
315  FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316    return FreeInst(std::move(__ptr));
317  }
318
319  std::string FreeInst::getAssembly() const noexcept {
320    return joinTokens(collectOpTokens("free"s, ptr));
321  }
322
323  //-----------------------------------------------------------
324  // class LoadInst
325  //-----------------------------------------------------------
326  LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                     ValueTy &&__ptr) noexcept
```

VARIABLES
- Local
  - obj: 0x0000000000242f90
  - tinfo: <invalid address>
  - dest: (libSCBackend.so`(anony…
  - globals: <variable not availa…
  - header: <variable not availab…
- Static
- Global
- Registers

WATCH

CALL STACK — PAUSED ON BREAKPOINT
- __cxxabiv1::__cxa_throw(void *,
- std::__cxx11::basic_string<char,
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(s
- result::Result<std::__cxx11::bas

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch

MODULES

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

Filter (e.g. text, !exclude)

```
Console is in 'commands' mode, prefix expressions with '?'.
Launching:                    swpp-compiler/build/swpp-compiler benchmarks/anagram/src/anagram.ll test.s
Launched process 2701418
```

SSH: | main* | ⊗ 0 ⚠ 0 ⓘ 2 | 0 | Debug (swpp-compiler) | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build [all] | [swpp-compiler] | Format: auto Disasm: auto Deref: on Console: cmd | Ln 77, Col 1 | Spaces: 2 | UTF-8 | LF | C++ | Linux

File Edit Selection View Go Run Terminal Help — eh_throw.cc - swpp-compiler - Visual Studio Code

RUN ... Debug

emitter.cpp | launch.json | eh_throw.cc

libstdc++-v3 > libsupc++ > eh_throw.cc > __cxa_throw(void *, std::type_info *, void(_GLIBCXX_CDTOR_

Variables in current stack entry

VARIABLES
- Local
  - obj: 0x0000000000242f90
  - tinfo: <invalid address>
  - dest: (libSCBackend.so`(anony…
  - globals: <variable not availa…
  - header: <variable not availab…
- Static
- Global
- Registers

WATCH

```
    __cxa_refcounted_exception *header
      = __get_refcounted_exception_header_from_obj (obj);
    header->referenceCount = 0;
    header->exc.exceptionType = tinfo;
    header->exc.exceptionDestructor = dest;
    header->exc.unexpectedHandler = std::get_unexpected ();
    header->exc.terminateHandler = std::get_terminate ();
    __GXX_INIT_PRIMARY_EXCEPTION_CLASS(header->exc.unwindHeader.exception_class);
    header->exc.unwindHeader.exception_cleanup = __gxx_exception_cleanup;

    return header;
  }

  extern "C" void
  __cxxabiv1::__cxa_throw (void *obj, std::type_info *tinfo,
          void (_GLIBCXX_CDTOR_CALLABI *dest) (void *))
  {
    PROBE2 (throw, obj, tinfo);

    __cxa_eh_globals *globals = __cxa_get_globals ();
    globals->uncaughtExceptions += 1;
    // Definitely a primary.
    __cxa_refcounted_exception *header =
      __cxa_init_primary_exception(obj, tinfo, dest);
    header->referenceCount = 1;

  #ifdef __USING_SJLJ_EXCEPTIONS__
    _Unwind_SjLj_RaiseException (&header->exc.unwindHeader);
  #else
    _Unwind_RaiseException (&header->exc.unwindHeader);
  #endif

    // Some sort of unwinding error.  Note that terminate is a handler.
```

inst.cpp

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

```
//-----------------------------------------------------------
// class MallocInst
//-----------------------------------------------------------
MallocInst::MallocInst(const GeneralRegister __target,
                       ValueTy &&__size) noexcept
    : AbstractInst(), target(__target), size(std::move(__size)) {}

MallocInst MallocInst::create(const GeneralRegister __target,
                              ValueTy &&__size) noexcept {
  return MallocInst(__target, std::move(__size));
}

std::string MallocInst::getAssembly() const noexcept {
  return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
}

//-----------------------------------------------------------
// class FreeInst
//-----------------------------------------------------------
FreeInst::FreeInst(ValueTy &&__ptr) noexcept
    : AbstractInst(), ptr(std::move(__ptr)) {}

FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
  return FreeInst(std::move(__ptr));
}

std::string FreeInst::getAssembly() const noexcept {
  return joinTokens(collectOpTokens("free"s, ptr));
}

//-----------------------------------------------------------
// class LoadInst
//-----------------------------------------------------------
LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
                   ValueTy &&__ptr) noexcept
```

Navigate call stack

CALL STACK — PAUSED ON BREAKPOINT
- __cxxabiv1::__cxa_throw(void *,
- std::__cxx11::basic_string<char,
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(s
- result::Result<std::__cxx11::bas

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch

MODULES

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

Filter (e.g. text, !exclude)

```
Console is in 'commands' mode, prefix expressions with '?'.
Launching:                    swpp-compiler/build/swpp-compiler benchmarks/anagram/src/anagram.ll test.s
Launched process 2701418
```

SSH: | main* | 0 0 0 2 | 0 | Debug (swpp-compiler) | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build [all] | [swpp-compiler] | Format: auto Disasm: auto Deref: on Console: cmd | Ln 77, Col 1 | Spaces: 2 | UTF-8 | LF | C++ | Linux

File Edit Selection View Go Run Terminal Help — eh_throw.cc - swpp-compiler — Visual Studio Code

RUN ... Debug

emitter.cpp | launch.json | eh_throw.cc | inst.cpp

libstdc++-v3 > libsupc++ > eh_throw.cc > __cxa_throw(void *, std::type_info *, void(_GLIBCXX_CDTOR_

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

Execute until next breakpoint

```
60  {
61    __cxa_refcounted_exception *header
62      = __get_refcounted_exception_header_fr
63    header->referenceCount = 0;
64    header->exc.exceptionType = tinfo;
65    header->exc.exceptionDestructor = dest;
66    header->exc.unexpectedHandler = std::get_unexpected ();
67    header->exc.terminateHandler = std::get_terminate ();
68    __GXX_INIT_PRIMARY_EXCEPTION_CLASS(header->exc.unwindHeader.exception_class);
69    header->exc.unwindHeader.exception_cleanup = __cxa_exception_cleanup;
70
71    return header;
72  }
73
74  extern "C" void
75  __cxxabiv1::__cxa_throw (void *obj, std::type_info *tinfo,
76          void (_GLIBCXX_CDTOR_CALLABI *dest) (void *))
77  {
78    PROBE2 (throw, obj, tinfo);
79
80    __cxa_eh_globals *globals = __cxa_get_globals ();
81    globals->uncaughtExceptions += 1;
82    // Definitely a primary.
83    __cxa_refcounted_exception *header =
84      __cxa_init_primary_exception(obj, tinfo, dest);
85    header->referenceCount = 1;
86
87  #ifdef __USING_SJLJ_EXCEPTIONS__
88    _Unwind_SjLj_RaiseException (&header->exc.unwindHeader);
89  #else
90    _Unwind_RaiseException (&header->exc.unwindHeader);
91  #endif
92
93    // Some sort of unwinding error.  Note that terminate is a handler.
94    __cxa_begin_catch (&header->exc.unwindHeader);
```

```
203  //----------------------------------------------------------
204  // class MallocInst
205  //----------------------------------------------------------
206  MallocInst::MallocInst(const GeneralRegister __target,
207                         ValueTy &&__size) noexcept
208      : AbstractInst(), target(__target), size(std::move(__size)) {}
209
300  MallocInst MallocInst::create(const GeneralRegister __target,
301                                ValueTy &&__size) noexcept {
302    return MallocInst(__target, std::move(__size));
303  }
304
305  std::string MallocInst::getAssembly() const noexcept {
306    return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
307  }
308
309  //----------------------------------------------------------
310  // class FreeInst
311  //----------------------------------------------------------
312  FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313      : AbstractInst(), ptr(std::move(__ptr)) {}
314
315  FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316    return FreeInst(std::move(__ptr));
317  }
318
319  std::string FreeInst::getAssembly() const noexcept {
320    return joinTokens(collectOpTokens("free"s, ptr));
321  }
322
323  //----------------------------------------------------------
324  // class LoadInst
325  //----------------------------------------------------------
326  LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
327                     ValueTy &&__ptr) noexcept
```

VARIABLES
- Local
  - obj: 0x0000000000242f90
  - tinfo: <invalid address>
  - dest: (libSCBackend.so`(anony…
  - globals: <variable not availa…
  - header: <variable not availab…
- Static
- Global
- Registers

WATCH

CALL STACK — PAUSED ON BREAKPOINT
- __cxxabiv1::__cxa_throw(void *,
- std::__cxx11::basic_string<char,
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(s

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch

MODULES

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
Console is in 'commands' mode, prefix expressions with '?'.
Launching:                    swpp-compiler/build/swpp-compiler benchmarks/anagram/src/anagram.ll test.s
Launched process 2701418
```

SSH: | main* | 0 0 0 2 | 0 | Debug (swpp-compiler) | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build [all] | [swpp-compiler] | Format: auto Disasm: auto Deref: on Console: cmd | Ln 77, Col 1 | Spaces: 2 | UTF-8 | LF | C++ | Linux

File Edit Selection View Go Run Terminal Help — eh_throw.cc - swpp-compiler - Visual Studio Code

RUN ... Debug

emitter.cpp | launch.json | eh_throw.cc | inst.cpp

libstdc++-v3 > libsupc++ > eh_throw.cc > __cxa_throw(void *, std::type_info *, void(_GLIBCXX_CDTOR_

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

Execute next line

```
60  {
61    __cxa_refcounted_exception *header
62      = __get_refcounted_exception_header_fr
63    header->referenceCount = 0;
64    header->exc.exceptionType = tinfo;
65    header->exc.exceptionDestructor = dest;
66    header->exc.unexpectedHandler = std::get_unexpected ();
67    header->exc.terminateHandler = std::get_terminate ();
68    __GXX_INIT_PRIMARY_EXCEPTION_CLASS(header->exc.unwindHeader.exception_class);
69    header->exc.unwindHeader.exception_cleanup = __cxa_exception_cleanup;
70
71    return header;
72  }
73
74  extern "C" void
75  __cxxabiv1::__cxa_throw (void *obj, std::type_info *tinfo,
76          void (_GLIBCXX_CDTOR_CALLABI *dest) (void *))
77  {
78    PROBE2 (throw, obj, tinfo);
79
80    __cxa_eh_globals *globals = __cxa_get_globals ();
81    globals->uncaughtExceptions += 1;
82    // Definitely a primary.
83    __cxa_refcounted_exception *header =
84      __cxa_init_primary_exception(obj, tinfo, dest);
85    header->referenceCount = 1;
86
87  #ifdef __USING_SJLJ_EXCEPTIONS__
88    _Unwind_SjLj_RaiseException (&header->exc.unwindHeader);
89  #else
90    _Unwind_RaiseException (&header->exc.unwindHeader);
91  #endif
92
93    // Some sort of unwinding error.  Note that terminate is a handler.
94    __cxa_begin_catch (&header->exc.unwindHeader);
```

```
203  //----------------------------------------------------------
204  // class MallocInst
205  //----------------------------------------------------------
206  MallocInst::MallocInst(const GeneralRegister __target,
                            ValueTy &&__size) noexcept
207    : Inst(), target(__target), size(std::move(__size)) {}
299
300  MallocInst MallocInst::create(const GeneralRegister __target,
                                   ValueTy &&__size) noexcept {
301    return MallocInst(__target, std::move(__size));
302  }
303
304  std::string MallocInst::getAssembly() const noexcept {
305    return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
306  }
307
308
309  //----------------------------------------------------------
310  // class FreeInst
311  //----------------------------------------------------------
312  FreeInst::FreeInst(ValueTy &&__ptr) noexcept
313      : AbstractInst(), ptr(std::move(__ptr)) {}
314
315  FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
316    return FreeInst(std::move(__ptr));
317  }
318
319  std::string FreeInst::getAssembly() const noexcept {
320    return joinTokens(collectOpTokens("free"s, ptr));
321  }
322
323  //----------------------------------------------------------
324  // class LoadInst
325  //----------------------------------------------------------
326  LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
                        ValueTy &&__ptr) noexcept
```

VARIABLES
- Local
  - obj: 0x0000000000242f90
  - tinfo: <invalid address>
  - dest: (libSCBackend.so`(anony…
  - globals: <variable not availa…
  - header: <variable not availab…
- Static
- Global
- Registers

WATCH

CALL STACK — PAUSED ON BREAKPOINT
- __cxxabiv1::__cxa_throw(void *,
- std::__cxx11::basic_string<char,
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(s
- result::Result<std::__cxx11::bas

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch

MODULES

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

Filter (e.g. text, !exclude)

```
Console is in 'commands' mode, prefix expressions with '?'.
Launching:                    swpp-compiler/build/swpp-compiler benchmarks/anagram/src/anagram.ll test.s
Launched process 2701418
```

SSH: | main* | 0 0 0 2 | 0 | Debug (swpp-compiler) | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build [all] | [swpp-compiler] | Format: auto Disasm: auto Deref: on Console: cmd | Ln 77, Col 1 | Spaces: 2 | UTF-8 | LF | C++ | Linux

Step In
(Get inside the function at current line &
execute the first line of that function)

Step out

(Execute until the end of current function & execute the next line of the caller)

File Edit Selection View Go Run Terminal Help — eh_throw.cc - swpp-compiler - Visual Studio Code

RUN ... Debug

emitter.cpp | launch.json | eh_throw.cc | inst.cpp

libstdc++-v3 > libsupc++ > eh_throw.cc > __cxa_throw(void *, std::type_info *, void(_GLIBCXX_CDTOR_

src > lib > backend > assembly > inst.cpp > {} sc > {} backend > {} assembly > {} inst

Restart / Stop debugging

```
{
  __cxa_refcounted_exception *header
    = __get_refcounted_exception_header_fr
  header->referenceCount = 0;
  header->exc.exceptionType = tinfo;
  header->exc.exceptionDestructor = dest;
  header->exc.unexpectedHandler = std::get_unexpected ();
  header->exc.terminateHandler = std::get_terminate ();
  __GXX_INIT_PRIMARY_EXCEPTION_CLASS(header->exc.unwindHeader.exception_class);
  header->exc.unwindHeader.exception_cleanup = __gxx_exc

  return header;
}

extern "C" void
__cxxabiv1::__cxa_throw (void *obj, std::type_info *tinfo,
        void (_GLIBCXX_CDTOR_CALLABI *dest) (void *))
{
  PROBE2 (throw, obj, tinfo);

  __cxa_eh_globals *globals = __cxa_get_globals ();
  globals->uncaughtExceptions += 1;
  // Definitely a primary.
  __cxa_refcounted_exception *header =
    __cxa_init_primary_exception(obj, tinfo, dest);
  header->referenceCount = 1;

#ifdef __USING_SJLJ_EXCEPTIONS__
  _Unwind_SjLj_RaiseException (&header->exc.unwindHeader);
#else
  _Unwind_RaiseException (&header->exc.unwindHeader);
#endif

  // Some sort of unwinding error.  Note that terminate is a handler.
  __cxa_begin_catch (&header->exc.unwindHeader);
```

```
//----------------------------------------------------------
:Inst(const GeneralRegister __target,
      ValueTy &&__size) noexcept
:(), target(__target), size(std::move(__size)) {}

MallocInst MallocInst::create(const GeneralRegister __target,
                              ValueTy &&__size) noexcept {
  return MallocInst(__target, std::move(__size));
}

std::string MallocInst::getAssembly() const noexcept {
  return joinTokens(prependTarget(target, collectOpTokens("malloc"s, size)));
}

//----------------------------------------------------------
// class FreeInst
//----------------------------------------------------------
FreeInst::FreeInst(ValueTy &&__ptr) noexcept
    : AbstractInst(), ptr(std::move(__ptr)) {}

FreeInst FreeInst::create(ValueTy &&__ptr) noexcept {
  return FreeInst(std::move(__ptr));
}

std::string FreeInst::getAssembly() const noexcept {
  return joinTokens(collectOpTokens("free"s, ptr));
}

//----------------------------------------------------------
// class LoadInst
//----------------------------------------------------------
LoadInst::LoadInst(const GeneralRegister __target, const AccessWidth __size,
                   ValueTy &&__ptr) noexcept
```

VARIABLES
- Local
  - obj: 0x0000000000242f90
  - tinfo: <invalid address>
  - dest: (libSCBackend.so`(anony…
  - globals: <variable not availa…
  - header: <variable not availab…
- Static
- Global
- Registers

WATCH

CALL STACK — PAUSED ON BREAKPOINT
- __cxxabiv1::__cxa_throw(void *,
- std::__cxx11::basic_string<char,
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(s
- result::Result<std::__cxx11::bas

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch

MODULES

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

Filter (e.g. text, !exclude)

```
Console is in 'commands' mode, prefix expressions with '?'.
Launching:                    swpp-compiler/build/swpp-compiler benchmarks/anagram/src/anagram.ll test.s
Launched process 2701418
```

SSH: | main* | 0 0 0 2 | 0 | Debug (swpp-compiler) | CMake: [Debug]: Ready | [Clang 14.0.0 x86_64-unknown-linux-gnu] | Build [all] | [swpp-compiler] | Format: auto Disasm: auto Deref: on Console: cmd | Ln 77, Col 1 | Spaces: 2 | UTF-8 | LF | C++ | Linux

Narrowing Down with Debugger

- How do you locate the exact line that crashes?
  - Debugger pinpoints (automatically pauses on) the crash site
  - No need to insert new code, rebuild, etc.

Traceback with Debugger

- You have to locate the code that first went wrong
  - Debugger shows you the call stack at the moment of the crash
  - Clicking is all you need to navigate through the call stacks
  - Find the first call stack with unexpected value or control flow

Traceback with Debugger

- Sometimes you have to monitor the change of values
  - If your code does not throw or crash, debugger won't pause
  - You can use breakpoint to pause execution at a certain point

File Edit Selection View Go Run Terminal Help — inst.cpp - swpp-compiler — Visual Studio Code

RUN ... Debug

VARIABLES

WATCH

CALL STACK

BREAKPOINTS
- C++: on throw
- C++: on catch
- inst.cpp src/lib/backend/asse... 21

MODULES

inst.cpp | result.h | CMakeLists.txt M

src > lib > backend > assembly > inst.cpp > {} `anonymous-namespace' > joinTokens(std::vector<std::string>&&)

Seunghyeon Nam, 2 days ago | 2 authors (You and others)

```
1  #include "inst.h"
2
3  #include "../../../result.h"
4
5  #include <algorithm>
6  #include <numeric>
7
8  using namespace std::string_literals;
9
Seunghyeon Nam, 2 days ago | 2 authors (You and others)
10 namespace {
11 std::string joinTokens(std::vector<std::string> &&__tokens) noexcept {
12   const auto collection_len = std::accumulate(
13       __tokens.cbegin(), __tokens.cend(), static_cast<size_t>(0),
14       [](const size_t len, const auto &line) -> size_t {
15         return len + line.size() + 1; // trailing whitespace or colon
16       });
17
18   std::string joined_string;
19   joined_string.reserve(collection_len);
20   for (const auto &token : __tokens) {
21     joined_string.append(token);        You, 2 months ago • Init …
22     joined_string.append(" ");
23   }
24   // remove trailing whitespace
25   joined_string.pop_back();
26   return joined_string;
27 }
28
29 using namespace sc::backend::assembly::inst;
30
31 std::string getToken(const IcmpCondition __cond) noexcept {
32   switch (__cond) {
33   case IcmpCondition::EQ:
```

Click on the left side of the line number

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

zsh

swpp-compiler main ±

SSH: main* 0 0 0 Debug (swpp-compiler) CMake: [Debug]: Ready [Clang 14.0.0 x86_64-unknown-linux-gnu] Build [all] [swpp-compiler] You, 2 months ago Ln 21, Col 33 Spaces: 2 UTF-8 LF C++ Linux

File Edit Selection View Go Run Terminal Help — inst.cpp - swpp-compiler — Visual Studio Code

RUN ... Debug

inst.cpp | result.h | CMakeLists.txt M

src > lib > backend > assembly > inst.cpp > {} `anonymous-namespace' > joinTokens(std::vector<std::string>&&)

Seunghyeon Nam, 2 days ago | 2 authors (You and others)

```
1  #include "inst.h"
2
3  #include "../../../result.h"
4
5  #include <algorithm>
6  #include <numeric>
7
8  using namespace std::string_literals;
9
Seunghyeon Nam, 2 days ago | 2 authors (You and others)
10 namespace {
11 std::string joinTokens(std::vector<std::string> &&__tokens) noexcept {
12   const auto collection_len = std::accumulate(
13       __tokens.cbegin(), __tokens.cend(), static_cast<size_t>(0),
14       [](const size_t len, const auto &line) -> size_t {
15         return len + line.size() + 1; // trailing whitespace or colon
16       });
17
18   std::string joined_string;
19   joined_string.reserve(collection_len);
20   for (const auto &token : __tokens) {
21     joined_string.append(token);        You, 2 months ago • Init …
22     joined_string.append(" ");
23   }
24   // remove trailing whitespace
26   joined_string.pop_back();
27   return joined_string;
28 }
29
30 using namespace sc::backend::assembly::inst;
31
32 std::string getToken(const IcmpCondition __cond) noexcept {
33   switch (__cond) {
34   case IcmpCondition::EQ:
```

Paused on breakpoint

VARIABLES
- Local
  - __tokens: size=3
  - collection_len: 14
  - joined_string: error: summary…
  - token: {_M_string_length:5}
- Static
- Global
- Registers

WATCH

CALL STACK — PAUSED ON BREAKPOINT
- (anonymous namespace)::joinToker
- sc::backend::assembly::inst::Fur
- sc::backend::emitter::AssemblyEm
- llvm::InstVisitor<sc::backend::e
- void llvm::InstVisitor<sc::backe
- llvm::InstVisitor<sc::backend::e
- sc::backend::emitAssembly[abi:c
- (anonymous namespace)::compile(

BREAKPOINTS
- [x] C++: on throw
- [ ] C++: on catch
- [x] inst.cpp src/lib/backend/asse… 21

MODULES

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

zsh
Debug

SSH: main* 0 0 0 Debug (swpp-compiler) CMake: [Debug]: Ready [Clang 14.0.0 x86_64-unknown-linux-gnu] Build [all] [swpp-compiler] Format: auto Disasm: auto Deref: on Console: cmd Ln 21, Col 26 Spaces: 2 UTF-8 LF C++ Linux